Báo cáo giao việc cải tiến Fashion Visual Search

Ngày lập: 28/10/2026 | **Phiên bản:** 1.0 | **Đối tượng:** nhóm 4 thành viên

Tài liệu bàn giao đầy đủ bối cảnh để mỗi thành viên có thể bắt đầu làm việc mà không cần đọc lịch sử trao đổi. Quyết định đã thống nhất là chia theo bốn module liên tiếp trong pipeline; mỗi người sở hữu code, kiểm thử, thực nghiệm và phần báo cáo của module mình.

Quyết định triển khai

M1 Tiền xử lý → M2 Phát hiện và phân vùng → M3 Trích đặc trưng → M4 Truy xuất và xếp hạng.

Giữ YOLO, FashionCLIP và PostgreSQL/pgvector hiện có. Cải tiến tập trung vào độ nhất quán đầu vào, vùng lấy đặc trưng, biểu diễn màu và chất lượng xếp hạng. Dữ liệu, đánh giá và viết báo cáo được chia đều, không có một người chỉ làm benchmark hoặc tích hợp hộ cả nhóm.

Thành viên	Module chịu trách nhiệm	Giá trị bàn giao
Người 1	M1 Chuẩn hóa và tiền xử lý	Ảnh hợp lệ, đúng hướng; nhánh tăng sáng có điều kiện
Người 2	M2 Phát hiện và phân vùng	Đúng món đồ, đúng ROI; mask có phạm vi áp dụng rõ
Người 3	M3 Trích đặc trưng	Vector và descriptor màu nhất quán cho query/catalog
Người 4	M4 Truy xuất và xếp hạng	Sản phẩm phù hợp, thứ hạng và quyết định từ chối rõ ràng

Tên thành viên chưa được cung cấp; nhóm điền tên tương ứng trước khi khởi động. Dự trù **40 giờ/người**, điều chỉnh phạm vi sau hai ngày dựa trên giờ thực tế.

Cách sử dụng tài liệu

Đọc các trang về bối cảnh và giao diện trước, sau đó đọc gói việc của mình. Dùng phần dữ liệu, metric, thực nghiệm và nghiệm thu như giao thức chung. Các kiểu dữ liệu, module mới và lệnh mới được đánh dấu là đề xuất, chưa phải tính năng đã được triển khai.

Đây là **tài liệu điều phối nội bộ**. Báo cáo đồ án nộp giảng viên vẫn phải rút gọn còn 4-10 trang theo đề bài [S1]. Tài liệu này không khẳng định hệ thống đã tăng metric.

Tra cứu nhanh: yêu cầu trang 2; hiện trạng trang 3; kiến trúc trang 4; gói M1-M4 trang 5-8; interface trang 9; dữ liệu trang 10; metric trang 11; thực nghiệm trang 12; công sức trang 13; tiến độ trang 14; môi trường trang 15; sản phẩm nộp trang 16; nguồn trang 17.

Yêu cầu đồ án và phạm vi đã chốt

Yêu cầu cần đáp ứng

Theo đề bài, lựa chọn phù hợp là **dự án xử lý ảnh thực tế**: bài toán thực tế, pipeline nhiều bước, có đối chứng và đánh giá định lượng. Cần dùng ít nhất một chương của học phần, ghi rõ phương pháp, tham số và lý do chọn; tối thiểu 50 ảnh đánh giá [S1, trang PDF 2, 3, 7].

Trang 7 ghi phần thử nghiệm áp dụng yêu cầu lựa chọn 1. Để bám sát đề, nhóm chuẩn bị 3-5 cấu hình/phương pháp so sánh, có đối chứng cổ điển, chạy trên cùng dữ liệu đầu vào. Các ablation bổ sung trình bày riêng, không gọi mọi biến thể tham số là một phương pháp SOTA.

Sản phẩm nộp	Yêu cầu từ đề bài
Báo cáo	PDF 4-10 trang; bài toán, dữ liệu, phương pháp, kết quả, thảo luận, phân công, khai báo AI
Slide	10-15 slide, PPTX và PDF; trình bày 10 phút, hỏi đáp 5 phút
Mã nguồn	Thư mục code, requirements.txt, README, 5-10 ảnh trong samples
Đóng gói	LopXX_NhomYY_TenDeTai.zip; trưởng nhóm đại diện nộp
Hạn nộp	11/11/2026; kiểm tra lại thông báo giảng viên nếu có cập nhật
Video	Khuyến khích, tối đa 3 phút; không là điều kiện bắt buộc

Kiến thức áp dụng

- **M1:** chương 2 về lấy mẫu, độ phân giải; chương 3 về gamma và tương phản.
- **M2:** chương 5 về phân ngưỡng/phân vùng; chương 7 về opening/closing.
- **M3:** chương 4 về HSV; chương 3 về histogram. Histogram HSV dùng cho retrieval là cách vận dụng, không phải toàn bộ thuật toán được dạy nguyên dạng trong slide.
- **M4:** so sánh đặc trưng, đối chứng và thiết kế thực nghiệm. Ranking hybrid là thiết kế của hệ thống, cần giải thích và kiểm chứng.

Chương 6 về nén ảnh được ghi nhận nhưng không nằm trong gói bắt buộc. CLAHE, GrabCut, HOG hoặc mô hình segmentation mới là phần mở rộng nếu dùng; không gán chúng thành nội dung trực tiếp của các slide đã cung cấp.

Giới hạn chung: không huấn luyện lại mô hình lớn, không xây lại ETL/DB/frontend, không cam kết tăng một tỷ lệ metric. Phương pháp không cải thiện vẫn có giá trị nếu phép thử đúng và phân tích được nguyên nhân.

Hiện trạng và vấn đề cần giải quyết

Bằng chứng kiểm tra ngày 28/10/2026

Hạng mục	Trạng thái quan sát được
Dữ liệu local	1.752 bản ghi metadata và 1.752 bản ghi có vector
Encoder trong file vector	patrickjohncyh/fashion-clip; vector 512 chiều
Trọng số detection	Có models/object-detection/best.pt trên máy làm việc
Bộ đánh giá hiện có	10 auto_smoke + 28 starter_catalog + 16 real_outfits
Kiểm thử	18 unit test hiện có chạy qua trong lượt lập báo cáo
Runtime và DB	Chưa xác minh lại end-to-end trong lượt lập báo cáo; số bản ghi local không phải số sản phẩm đang phục vụ

Nguồn là code và file trên workspace [S8-S12]. README còn nêu catalog 850; số đó không đại diện snapshot file hiện tại. Chưa có bằng chứng cải tiến của M1-M4 vì đây là kế hoạch giao việc.

Kết quả đã lưu chỉ dùng để định hướng

Report ngày **30/04/2026** ghi real_outfits có 16 query: detection_success_rate 0,9375; category_at_5 0,875; color_at_5 0,50; Recall/MRR chưa có nhãn để tính. Starter_catalog có Color@5 khoảng 0,5357. Điểm tổng hợp 0,8152 là công thức nội bộ và các gate tổng thể FAIL; không phải accuracy 81,52% [S12].

Các con số này chưa được chạy lại trên trạng thái code hiện tại. Detection success chỉ nghĩa là có ít nhất một detection; không tương đương detection recall hoặc mAP.

Các lỗi và giới hạn đã biết

1. Bộ lọc màu loại pixel sáng, ít bão hòa như nền, có thể loại luôn vải trắng. Phản ví dụ tổng hợp 96% trắng và 4% màu da ở tâm được gán khaki; đây không phải metric ảnh thật.
2. ROI hình chữ nhật có thể lẫn da, nền và món khác; crop trung tâm chưa bảo đảm là vải.
3. Evaluator gom sản phẩm từ nhiều detection; đúng loại và đúng màu có thể đến từ hai kết quả khác nhau.
4. Nhãn màu starter_catalog được lấy từ metadata hệ thống, cần người kiểm tra độc lập.
5. Có 54 query trong ba bộ nhưng chỉ 16 outfit thật. Chưa đủ cơ sở kết luận chất lượng ngoài thực tế.
6. Kết quả retrieval phụ thuộc catalog: không có sản phẩm phù hợp hoặc không lấy được vào candidate pool thì rerank tốt hơn vẫn không giải quyết được.

Kiến trúc và nguyên tắc tích hợp

Luồng hiện tại

Upload vào FastAPI chính tại cổng 8000 → gọi dịch vụ detection cổng 8001 → YOLO → crop tight/padded/wide → JPEG/base64 → API chính giải mã → FashionCLIP → pgvector → rerank → React tại cổng 5173.

Đây là luồng **hai service**, không thể truyền trực tiếp NumPy array từ detector sang API bằng lời gọi hàm thông thường. DTO trong tài liệu mô tả hợp đồng logic; qua HTTP phải encode/decode rõ ràng.

Luồng mục tiêu

Công đoạn	Nơi thực thi dự kiến	Bàn giao
M1	Bên trong detection service	source_rgb, detection_rgb và metadata
M2	Detection service, sau M1	GarmentROI; serialize qua response detection
M3	API chính và xử lý catalog offline	FeatureBundle có embedding/descriptor
M4	API chính, DB và cache descriptor	top-K, score breakdown, trạng thái từ chối

M1 → M2 trong bộ nhớ. M2 → M3 qua HTTP. M3 → M4 trong API chính. Giữ API frontend đang dùng; bổ sung trường tùy chọn thay vì xóa/đổi nghĩa trường cũ.

Tách nhánh ảnh để bảo toàn màu

Sau sửa EXIF và resize hình học, giữ source_rgb. Tạo detection_rgb bằng gamma trên một bản sao; hai nhánh cùng kích thước. YOLO chạy trên detection_rgb, nhưng ROI và màu lấy từ source_rgb. Gamma không thay đổi tọa độ.

M3 giữ processor cuối của FashionCLIP. Màu được tính trên ROI gốc và mask, không trên padding trắng hoặc ảnh đã gamma. Đầu vào RGB phải được chuyển sang BGR chỉ ở điểm thư viện yêu cầu; tránh đảo kênh hai lần.

Luồng catalog offline

Dùng cùng phiên bản M1/M2/M3 để tạo đặc trưng phù hợp cho catalog. Nếu không có detection, dùng fallback đã chốt và ghi embedding_source/roi_source. M4 đọc index/cache tương thích phiên bản. Không ghi đè file vector/cache gốc khi đang thử.

Ranh giới code: M1 để thuật toán trong file riêng; M2 sở hữu detect/ROI trong detector.py; M3 sở hữu encoder/descriptor; M4 sở hữu các điểm gọi trong search_service.py. Hai chủ module cùng kiểm tra mỗi lần bàn giao.

Gói việc M1 cho người 1

Nhiệm vụ: chuẩn hóa đầu vào và thử tiền xử lý thiếu sáng có điều kiện. **Ngân sách:** 40 giờ gồm phần chung. **Kiến thức:** [S2, trang 23-37], [S3, trang 18-21].

Đọc trước và vị trí code

Đọc object_detection/detector.py: detect_from_bytes, _process, _make_crop_variant; object_detection/api.py: detect_from_upload. Hiện ảnh được OpenCV giải mã trực tiếp; M1 cần xử lý EXIF đúng trước khi bàn giao M2.

Tạo AI/image_preprocessing.py hoặc tên tương đương đã thống nhất. Thuật toán đặt trong module riêng để không sửa chồng phần detect/ROI của người 2.

Công việc bắt buộc

1. **M1-01:** chốt kiểu ảnh RGB uint8, quy tắc alpha/EXIF và kích thước làm việc; ghi cấu hình thay vì số ẩn trong code.
2. **M1-02:** chuẩn hóa hình học một lần; giữ tỉ lệ; lưu kích thước gốc/chuẩn hóa và hệ số quy đổi. Không coi phóng lớn là phục hồi chi tiết.
3. **M1-03:** tính thống kê độ sáng và gamma có điều kiện. Với s = r^gamma trên r thuộc [0,1], ghi đúng quy ước gamma; không trộn với API dùng số nghịch đảo.
4. **M1-04:** trả hai bản source_rgb/detection_rgb cùng hình học; không sửa source_rgb tại chỗ.
5. **M1-05:** viết unit test và phép thử bật/tắt trên validation; ghi ca tăng sáng làm xấu detection/retrieval.
6. **M1-06:** bàn giao fixture ảnh tối/đủ sáng, metadata và lệnh chạy test; cùng M2 kiểm tra bbox sau biến đổi.

Tiêu chí nghiệm thu

- EXIF được áp dụng đúng một lần; ảnh RGB/RGBA/gray có kết quả theo quy tắc công bố.
- Hai nhánh cùng shape; tọa độ bbox khớp nguồn sau chuẩn hóa; lỗi decode trả lỗi có kiểm soát.
- Ảnh đủ sáng không bị biến đổi nếu điều kiện không kích hoạt; source_rgb không bị gamma ghi đè.
- Có bảng latency tiền xử lý, tỷ lệ bật gamma, và kết quả riêng ảnh tối/đủ sáng với cùng protocol.

Giá trị kỳ vọng: độ bền trên ảnh thiếu sáng. Không mặc định ảnh nhìn sáng hơn sẽ tăng FashionCLIP hoặc mọi metric.

Ngoài phạm vi: CLAHE, nhiều bộ lọc, tối ưu JPEG, fine-tune. Khi còn thời gian mới thử một mở rộng; không dồn thêm thuật toán để đủ số chương.

Ba giờ đầu: tái hiện decode hiện tại → tạo fixture EXIF/tối/sáng → chốt DTO với người 2 → chạy được normalize_image không cần YOLO.

Gói việc M2 cho người 2

Nhiệm vụ: phát hiện quần áo, cắt ROI và cung cấp mask có giới hạn rõ. **Ngân sách:** 40 giờ. **Kiến thức:** [S5, trang 61-70, 80-85], [S7, trang 28-38].

Đọc trước và vị trí code

Đọc ClothingDetector._process/_make_crop_variant, object_detection/schemas.py, utils.py:crops_to_response. Hiện code chỉ dùng result.boxes, chưa có mask; crop được pad trắng rồi resize 224. Giữ best.pt hiện có, không mở thêm bài toán huấn luyện.

Công việc bắt buộc

1. **M2-01:** dùng detection_rgb từ M1 để chạy YOLO; clip bbox vào biên, loại box không hợp lệ; giữ class/confidence theo schema hiện có.
2. **M2-02:** crop từ source_rgb cùng tọa độ; giữ ROI gốc trước padding/resize để trích màu. Gắn garment_id ổn định trong một query.
3. **M2-03:** chọn một phương pháp mask cổ điển cho điều kiện phù hợp. Otsu cần độ sáng hai miền có khả năng tách; phát triển vùng cần seed/quy tắc tương đồng được giải thích.
4. **M2-04:** thử opening/closing nhẹ sau mask thô; cấu trúc/kích thước chọn trên validation. Không fill mọi lỗ hoặc giữ component lớn nhất một cách mặc định.
5. **M2-05:** khi mask lỗi, trả mask=None và roi_status/fallback_reason. Tỷ lệ diện tích hợp lý chỉ là kiểm tra hình học, không chứng minh ngữ nghĩa mask đúng.
6. **M2-06:** bổ sung ROI nguồn và mask qua response; RGB nguồn ưu tiên PNG cho nhánh màu, mask PNG nhị phân; giữ trường JPEG/base64 cũ cho tương thích.

Tiêu chí nghiệm thu

- Bbox, ROI, mask đúng hệ tọa độ; mask decode ra đúng kích thước ROI, chỉ chứa 0/1 hoặc 0/255 theo hợp đồng.
- Ca bbox chạm biên, box nhỏ, không phát hiện, mask rỗng và mask bất thường đều có kiểm thử.
- Có hình mask thô/sau morphology/ground truth, số liệu IoU trên tập có mask và tỷ lệ fallback.
- Kết quả màu/retrieval được đo khi chỉ thay mask; không đổi đồng thời model và trọng số ranking.

Giá trị kỳ vọng: giảm nền/da trong nhánh màu. Morphology không tự biết đâu là áo; áo trắng trên nền trắng và người mặc nhiều lớp vẫn là ca khó.

Ngoài phạm vi: segmentation tổng quát, SAM/U-Net, huấn luyện YOLO, thay nền embedding. Giới hạn mask phục vụ màu để kiểm soát độ khó.

Ba giờ đầu: nhận fixture M1 → chạy crop với bbox giả lập → giữ ROI gốc → trả mask=None đúng schema trước khi triển khai phân vùng.

Gói việc M3 cho người 3

Nhiệm vụ: tạo đặc trưng hình ảnh/màu nhất quán cho query và catalog. **Ngân sách:** 40 giờ. **Kiến thức:** [S4, trang 37-41], [S3, trang 23-38].

Đọc trước và vị trí code

Đọc AI/color_utils.py: extract_dominant_color, name_color, color_similarity; AI/embedding_model.py: encode_image/get_embedding_config; AI/vectorizer.py: select_embedding_image/process_single_item. Trong API, _search_single_crop_variant đang lấy màu và vector từ cùng ảnh crop.

Công việc bắt buộc

1. **M3-01:** giữ FashionCLIP và vector 512 chiều, chuẩn hóa L2; phát hiện NaN/Inf/sai chiều hoặc thất bại tải encoder.
2. **M3-02:** lấy màu từ ROI nguồn, loại padding theo metadata/mask thay vì loại toàn bộ pixel trắng; xử lý mask=None và mask rỗng theo fallback.
3. **M3-03:** tạo histogram H-S cho vùng có màu, nhánh vô sắc dùng thông tin V để phân biệt trắng/xám/đen. Hue có tính vòng tròn và kém tin cậy khi saturation thấp.
4. **M3-04:** chốt số bin, chuẩn hóa, xử lý vùng rỗng và phép so sánh được M4 dùng. Đây là thiết kế cần validation, không gọi một bộ tham số là tối ưu trước khi đo.
5. **M3-05:** cung cấp một hàm tạo FeatureBundle cho query/catalog. Version gồm model/version, roi_policy và color_descriptor_version; ghi cache thử nghiệm riêng.
6. **M3-06:** giữ descriptor cũ để đối chứng; bàn giao script tạo cache và fixture đặc trưng cho M4. Không sửa trọng số ranking trong module này.

Tiêu chí nghiệm thu

- Test vải trắng có vùng da nhỏ, màu vô sắc, nhiều màu, crop hẹp/padding và mask lỗi.
- Vector đúng chiều, hữu hạn, chuẩn hóa; histogram không âm, tổng bằng 1 khi có pixel hợp lệ; quy ước empty được ghi rõ.
- Query/catalog cùng phiên bản; cache cũ không bị đọc nhầm; không ghi đè file vector gốc.
- Có bảng độ chính xác màu theo nhãn người kiểm tra, latency và tác động retrieval cũ/mới với bbox/ứng viên cố định.

Giá trị kỳ vọng: biểu diễn màu tốt hơn và ổn định khi bàn giao ranking. Đổi descriptor màu không bắt buộc tạo lại toàn bộ embedding; chỉ tạo lại embedding khi preprocessing/ROI của nhánh encoder thay đổi.

Ngoài phạm vi: fine-tune, HOG/Lab/DeltaE, nhiều họ descriptor, ranking hoặc migration DB lớn. Histogram-only là đối chứng cổ điển đủ để triển khai gói đầu.

Ba giờ đầu: chạy phản ví dụ màu trắng → định nghĩa FeatureBundle với M4 → tạo fixture vector/histogram không cần DB → chốt policy mask với M2.

Gói việc M4 cho người 4

Nhiệm vụ: truy xuất ứng viên, xếp hạng và quyết định trả kết quả. **Ngân sách:** 40 giờ. **Cơ sở:** đặc trưng từ M3, đối chứng và đánh giá định lượng [S1].

Đọc trước và vị trí code

Đọc db/database.py: search_similar_by_vector; api/search_service.py: rerank_products, merge_variant_rankings, apply_match_threshold, _search_single_crop_variant. Đọc api/schemas.py và endpoint /api/search/upload trước khi đổi response.

Hiện retrieval giới hạn tối đa 100 ứng viên mỗi variant; trọng số mặc định ảnh/loại/màu/text-style là 0,60/0,15/0,15/0,10. Ngưỡng mặc định 0,70 và image similarity 0,60 là cấu hình đang có, không phải giá trị tối ưu đã chứng minh.

Công việc bắt buộc

1. **M4-01:** nhận FeatureBundle, kiểm tra model/descriptor version; giữ vector retrieval pgvector và lưu số ứng viên thực tế.
2. **M4-02:** dùng descriptor màu mới trong rerank; giữ score breakdown ảnh/loại/màu/text-style/phạt theo cấu hình. Ghi rõ cách chuyển khoảng cách histogram thành score.
3. **M4-03:** triển khai ba chế độ histogram-only, embedding-only, hybrid. Histogram-only thực sự tìm trên catalog, không lấy trước ứng viên bằng CLIP rồi gọi đó là baseline cổ điển độc lập.
4. **M4-04:** chọn trọng số/ngưỡng trên validation; ghi trạng thái từ chối và lý do. Điểm tương đồng không được hiển thị như xác suất đúng đã hiệu chuẩn.
5. **M4-05:** ghép điểm gọi M3/M4 trong API; thêm trường tùy chọn, giữ contract frontend. Phân biệt lỗi service, không có detection, không có ứng viên và từ chối match.
6. **M4-06:** đo HitRate/MRR, joint loại+màu, latency và độ bao phủ kết quả; phân tích lỗi candidate recall so với lỗi rerank.

Tiêu chí nghiệm thu

- Test vector sai chiều/version, catalog/cache thiếu, không ứng viên, điểm thấp, hòa điểm và kết quả trùng từ nhiều crop.
- Ba chế độ chạy lại được trên cùng query/catalog/top-K; công bố khác biệt cơ chế tạo ứng viên.
- Trường similar_products và low_confidence_products giữ đúng nghĩa; không đổi một kết quả bị từ chối thành thành công trong metric end-to-end.
- Có bảng trước/sau ranking, cấu hình đã khóa và kiểm tra response API với frontend hiện có.

Ngoài phạm vi: viết lại DB/ETL/frontend, migration lớn, gánh toàn bộ evaluator hoặc tích hợp. **Ba giờ đầu:** chạy ranking bằng feature/product giả lập → chốt descriptor với M3 → lưu cấu hình gốc làm baseline.

Hợp đồng bàn giao dữ liệu

Các hợp đồng dưới đây là **đề xuất triển khai**, chưa tồn tại đầy đủ trong code. Bốn người chốt trong ngày đầu và lưu cùng schema_version. Tên hàm có thể đổi; ý nghĩa dữ liệu phải giữ nhất quán.

Hợp đồng	Trường và bất biến bắt buộc
PreparedImage M1→M2	source_rgb và detection_rgb: H×W×3, uint8 RGB, cùng shape; geometry_meta và quality_meta
GarmentROI M2→M3	query_id/garment_id, bbox_xyxy, class/confidence, ROI RGB nguồn, mask tùy chọn, roi_source, fallback_reason
FeatureBundle M3→M4	embedding 512, model/version, color descriptor/version, roi_policy, garment_id; quy ước descriptor rỗng
SearchResult M4→API	sản phẩm/id/rank, score breakdown, accepted/rejected, reason, config_id và thời gian

Tọa độ và màu

- Bbox dùng [x1,y1,x2,y2], x2/y2 loại trừ theo slicing; clip vào ảnh nguồn sau chuẩn hóa. Không trộn với tọa độ ảnh upload chưa sửa EXIF.
- Mask nằm trong hệ tọa độ **ROI nguồn**, không phải toàn ảnh hay crop 224. ROI rộng w, cao h thì mask shape h×w. Resize mask bằng nearest-neighbor nếu cần.
- Lưu kích thước ROI gốc, bbox, padding và biến đổi. Nhánh encoder có thể pad/resize riêng; nhánh màu lấy pixel ROI/mask trước bước đó.
- Mã màu RGB được công bố ở biên module. OpenCV BGR phải chuyển đúng một lần tại điểm phù hợp.

Qua biên HTTP

Trong bộ nhớ dùng array/đối tượng Python; qua detection response dùng trường tùy chọn như source_crop_png_base64, mask_png_base64 và roi_metadata. PNG giữ giá trị pixel tốt hơn việc nén JPEG lặp lại cho nhánh màu; đo thêm chi phí truyền.

mask_png_base64 giải mã thành mask 0/255; chuyển boolean ở nơi dùng. Không truyền NumPy array thẳng trong JSON. Không nhầm raw base64 với data URL; chốt một quy ước và test round-trip.

M2 sửa schema/serializer, M3 sửa decoder; cùng viết fixture round-trip. Giữ image_base64/crop_variants cũ trong thời gian chuyển tiếp. Khi trường mới vắng mặt, có fallback rõ và ghi chế độ xử lý vào report.

Cache và cấu hình

Cache gắn product_id + model/version + roi_policy + descriptor_version; không chỉ key bằng product_id. Cấu hình thử nghiệm và baseline tách file. Trường mismatch phải fail rõ hoặc dùng fallback công bố, không âm thầm trộn phiên bản.

Dữ liệu và quy trình gán nhãn

Phân bổ ngang nhau

Mỗi người chuẩn bị 20 ảnh outfit: 5 validation và 15 test. Sau khử trùng, nhóm có **20 validation + 60 test**. Có thể dùng lại ảnh hiện có nếu phù hợp và biết lịch sử sử dụng; 16 outfit cũ đã được tune nhiều lần nên ưu tiên validation/regression, không gọi là test mới độc lập.

Mỗi ảnh chọn một món mục tiêu để kiểm soát công sức. Kết luận chỉ áp dụng phạm vi này, không suy rộng thành tìm đúng mọi món trên ảnh nhiều người. Ghi nguồn, giấy phép/quyền sử dụng, nhóm ánh sáng và loại quần áo.

Mỗi người gán thêm 5 mask tham chiếu (2 validation + 3 test), tổng 20 mask. Đây là bộ phụ trợ nhỏ cho mask IoU, không phải benchmark segmentation mạnh.

Nhãn tối thiểu đề xuất

```
{
  "query_id": "q001",
  "split": "test",
  "image": "queries/test/q001.jpg",
  "target_id": "q001_top",
  "bbox_xyxy": [40, 60, 260, 340],
  "class_name": "short_sleeve_top",
  "color_names": ["white"],
  "relevance_status": "has_match",
  "acceptable_product_ids": ["PRODUCT_A"],
  "lighting_group": "normal"
}
```

Ví dụ là schema mới cần adapter cho labels hiện tại; không đưa thẳng vào runner cũ rồi cho rằng đã hỗ trợ. Bbox nhãn phải có quy tắc EXIF/hình học thống nhất với evaluator. Mask path/source/license được bổ sung khi có.

Kiểm nhãn và chống rò rỉ

Kiểm chéo theo vòng 1→2→3→4→1. Nhãn màu được người xem ảnh xác nhận, không lấy dominant_color_name do hệ thống sinh. Relevance dựa trên tiêu chí cố định: loại phù hợp, màu chủ đạo, kiểu dáng/họa tiết trong phạm vi đề tài.

Chọn acceptable IDs độc lập với một cấu hình đang tối ưu; kiểm tra nhiều sản phẩm ứng viên theo quy trình chung. Nếu danh sách relevance chưa đầy đủ, ghi rõ metric đo theo **tập sản phẩm đã gán nhãn**, không gọi là recall tuyệt đối trên toàn catalog. Sản phẩm chưa được xét không tự động là negative đã xác nhận.

Ca không có sản phẩm phù hợp phải có relevance_status=no_match được người kiểm tra; danh sách ID rỗng chưa đủ chứng minh no_match. Chia split theo ảnh nguồn/người/cảnh khi phù hợp để tránh bản crop hoặc ảnh gần trùng xuất hiện ở cả hai tập.

Giao thức metric và trách nhiệm đánh giá

Đơn vị đánh giá

Một query gồm một món mục tiêu có bbox/class. Ghép detection đúng class theo ánh xạ đã khóa và IoU với target; thử ngưỡng 0,5 làm cấu hình ban đầu, ghi rõ ngưỡng cuối. Mỗi target ghép tối đa một detection. Không lấy sản phẩm của detection khác để tính trúng.

Metric	Định nghĩa dùng trong báo cáo
Target detection recall	Số target ghép đúng / số target có nhãn; không thay bằng có ít nhất một box
HitRate@5	Tỷ lệ query has_match có ít nhất một acceptable ID trong top-5 của target đúng
MRR@5	Trung bình 1/rank của acceptable ID đầu tiên trong top-5; không có thì 0
Joint Category+Color@5	Có cùng một sản phẩm trong top-5 đúng cả loại và màu theo nhãn độc lập
Mask IoU	Diện tích giao / hợp giữa mask dự đoán và mask tham chiếu cùng tọa độ
Coverage	Tỷ lệ target/query được hệ thống chấp nhận trả match theo ngưỡng đã khóa
False acceptance no_match	Tỷ lệ ca no_match vẫn được hệ thống chấp nhận gợi ý
Latency	Thời gian M1/M2/M3/M4 và end-to-end; báo trung bình và p95 nếu đủ mẫu

Quy tắc đếm

- Không phát hiện đúng target thì end-to-end HitRate/MRR/joint của query has_match bằng 0; không loại query khó khỏi mẫu số.
- Tách chất lượng retrieval **trước ngưỡng** khỏi chất lượng **sau quyết định chấp nhận**. Kết quả bị từ chối có thể dùng chẩn đoán retrieval, nhưng không được tính như kết quả đã trả thành công.
- Query no_match không đưa vào mẫu số MRR/HitRate has_match; báo riêng false acceptance và số lượng mỗi nhóm.
- MRR chỉ xem top-5 phải gọi MRR@5. HitRate không phải Recall khi có nhiều relevant IDs; nếu tính Recall@5 thật, cần số relevant được xác nhận và công bố độ đầy đủ nhãn.
- Mask fallback không bị bỏ âm thầm khỏi thống kê. Báo IoU trên ca có mask dự đoán và tỷ lệ bao phủ/fallback; có thể thêm điểm 0 cho thất bại theo policy công bố.

Chia phần sửa evaluator

Người 1 lo manifest/split và timing; người 2 lo target matching/IoU; người 3 lo kiểm nhãn màu và joint metric; người 4 lo HitRate/MRR/rejection. Mỗi người viết test cho phần mình, không sửa đồng thời cùng file khi chưa chốt giao diện hàm.

Phản ví dụ bắt buộc: áo đen và váy trắng **không** thành áo trắng; có box sai món không thành detection đúng; MRR top-5 không được gọi MRR toàn bộ. Sau sửa, chạy lại baseline bằng evaluator mới trước khi so sánh.

Thiết kế thực nghiệm và đối chứng

Hai câu hỏi phải tách

So sánh phương pháp retrieval: histogram-only tìm trên toàn catalog, embedding-only dùng vector retrieval, hybrid dùng candidate + rerank. Giữ cùng catalog/query/target ROI/top-K và mô tả chi phí khác nhau. Không tiền lọc histogram-only bằng CLIP rồi gọi nó là đối chứng cổ điển độc lập.

So sánh rerank: đóng băng bbox và danh sách ứng viên CLIP cho từng query rồi thay descriptor/trọng số. Khi đó chênh lệch thể hiện tác dụng xếp hạng. Candidate HitRate@100 cho biết mức trần của bước rerank với pool đó.

Mã	Cấu hình	Câu hỏi và chủ trì
B0	Pipeline hiện tại, evaluator mới	Mốc chung; cả nhóm chạy lại
B1	B0 + M1 gamma có điều kiện	Ảnh tối có tốt hơn, ảnh sáng có xấu đi? Người 1
B2	Chỉ thêm mask thô cho màu	Vùng chọn pixel có giúp? Người 2 và 3
B3	B2 + morphology	Opening/closing có thêm lợi ích? Người 2
B4	Sửa màu trắng, rồi HSV	Descriptor tốt hơn hay chỉ thay nhãn? Người 3
C1	Histogram-only	Đối chứng cổ điển trên catalog chung; người 4 dùng descriptor người 3
C2	Embedding-only	Mốc học sâu không rerank
C3	Hybrid	Kết hợp ảnh/loại/màu và text-style theo cấu hình công bố
F	Các module được chọn trên validation	Chạy test cuối một lần sau khóa cấu hình

B1-B4 là ablation; C1-C3 là so sánh phương pháp/cấu hình retrieval. Báo cáo đồ án có thể trình bày C1-C3 cùng B0/F trong bảng chính và tách bảng ablation.

Điều kiện công bằng

Khóa model/trọng số, index/catalog snapshot, query split, quy tắc nhãn, top-K, số crop, candidate budget và ngưỡng khi so sánh tương ứng. Một thay đổi số crop/ứng viên phải ghi riêng, không gán lợi ích cho thuật toán khác.

Tách cold start với warm run; không gộp lần tải model vào latency ổn định. Ghi phần cứng và trạng thái cache. Chạy thử lỗi trước, không bỏ query lỗi khỏi kết quả cuối mà không giải thích.

Mỗi chủ module tự lưu CSV/JSON, cấu hình, ví dụ đúng/sai và kết luận. Chưa có kết quả thì ghi chưa đo; không điền phần trăm kỳ vọng như kết quả. Tập test không dùng chọn gamma, kernel, bins, trọng số hoặc ngưỡng.

Phân bổ công sức và phối hợp

Ngân sách dự kiến cho mỗi người

Hạng mục	Giờ/người	Đầu ra
Đọc code/tài liệu, thiết kế/interface	4	Hợp đồng và fixture
Cài đặt module	12	Module chạy độc lập
Kiểm thử và tích hợp	6	Test và bàn giao cặp module
Thực nghiệm và phân tích	8	Cấu hình, số liệu, ca thất bại
Dữ liệu và kiểm nhãn	6	20 ảnh và phần kiểm chéo
Báo cáo và slide	4	Khoảng 1 trang nội dung, 2 slide
Tổng	**40**	**160 giờ toàn nhóm**

Giả định kinh nghiệm Python tương đương và có môi trường cơ bản. Đây là dự trù, không phải bảo đảm giờ thực tế. Theo dõi công sức và độ khó sau hai ngày; không cân bằng bằng số hàm hay số dòng code.

Quy tắc điều chỉnh phạm vi

Giới hạn M2 ở mask có điều kiện; M3 dùng cache riêng và encoder cũ; M4 mock DB/snapshot khi hạ tầng chặn. Nếu M1 nhẹ hơn, bổ sung kiểm thử EXIF/hình học và timing. Không mở thêm model, migration hoặc frontend để kéo dài phạm vi.

Không chuyển việc chưa xong mặc định sang người giỏi hơn hoặc người tích hợp. Mọi thay đổi người phụ trách phải cập nhật task và giờ.

Trách nhiệm bàn giao

Bàn giao	Người tạo	Người nhận và cùng nghiệm thu
PreparedImage và hình học	1	2
ROI/mask và HTTP fixture	2	3
FeatureBundle/cache version	3	4
SearchResult/API	4	Nhóm kiểm tra với frontend
Dữ liệu/metric/report tổng hợp	Cả bốn	Kiểm chéo theo vòng

Mỗi người sở hữu phần sửa evaluator gắn module mình. Trưởng nhóm điều phối mốc và đóng gói, không nhận thay phần thuật toán hoặc viết toàn bộ báo cáo.

Tiến độ và cổng nghiệm thu

Mốc	Thời gian	Điều kiện hoàn thành
G0 Chốt giao diện	28-29/10	DTO, hình học, schema nhãn, fixtures, catalog/split dự kiến
G1 Module độc lập	30/10-03/11	Mỗi người có code, test, fixture và lượt validation
G2 Tích hợp	04-06/11	M1→M2, M2→M3 qua HTTP, M3→M4; khóa cấu hình, chạy test
G3 Tài liệu	07-09/11	Nội dung từng người, slide, bảng số liệu và ca lỗi
G4 Chạy lại	10/11	Máy khác/môi trường sạch làm theo README; kiểm tra gói nộp
Nộp	11/11	Trưởng nhóm nộp theo yêu cầu môn học

Gói bàn giao của mỗi module

1. Mã nguồn và danh sách file sửa; mô tả ngắn vấn đề, phương pháp và giới hạn.
2. Interface đầu vào/đầu ra, phiên bản và ít nhất một fixture chạy độc lập.
3. Lệnh kiểm thử, kết quả và các edge case bắt buộc của gói việc.
4. Cấu hình thực nghiệm, snapshot dữ liệu/model/cache, CSV/JSON và nhận xét.
5. Một số ảnh thành công/thất bại có chú thích; không chỉ chọn ảnh đẹp.
6. Phần báo cáo/slide, nguồn học thuật và khai báo công cụ AI đã dùng.

Cổng chất lượng chung

Không có lỗi làm sai tọa độ/kênh màu, không trộn model/descriptor version, không phá contract API cũ. Unit test cũ và test mới liên quan phải qua; có test tích hợp biên HTTP và kết quả API. Không cần tự đặt số lượng test để đủ quota.

Một module có thể không tăng metric và vẫn được nghiệm thu nếu chạy đúng, có đối chứng công bằng, thời gian trong phạm vi và kết luận trung thực. Nếu có regression trên nhóm quan trọng, cấu hình đó không trở thành mặc định cuối chỉ vì điểm trung bình tăng.

Khi bị chặn

Trong cập nhật ngắn, ghi: task ID, đã làm, bằng chứng/lệnh, đầu vào đang thiếu, cách tiếp tục với fixture và người cần phối hợp. Không chờ im lặng công đoạn trước; chủ động dùng mock theo contract đã chốt.

Mỗi ngày cập nhật trạng thái chưa làm/đang làm/đang review/hoàn thành, số giờ đã dùng và việc còn lại. Mọi thay đổi phạm vi lưu trong file giao việc, không chỉ nhắn miệng.

Môi trường và cách bắt đầu

Chạy hệ thống hiện có

Các lệnh dưới đây dựa trên cấu trúc repo hiện tại. Chạy từ thư mục gốc dự án; không đưa khóa R2/Apify hoặc nội dung .env vào báo cáo/Git.

```
if (-not (Test-Path .env)) {
    Copy-Item .env.example .env
}
# Điền cấu hình local trước khi chạy.
docker compose up --build
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:8001/health
Invoke-RestMethod 'http://localhost:8000/api/products?limit=1&offset=0'
```

Chỉ copy .env khi chưa tồn tại. Kiểm tra detection model_available, kết nối DB và số sản phẩm; health API chính thành công chưa chứng minh DB/encoder hoạt động đầy đủ. Frontend: localhost:5173; DB từ host: localhost:5433; trong Compose: postgres-pgvector:5432.

Trọng số cần ở models/object-detection/best.pt. Encoder có thể cần tải/cache Hugging Face; ảnh sản phẩm có thể cần quyền truy cập R2. Ghi rõ cách tải dữ liệu/model hợp lệ trong README gói nộp.

Kiểm thử và đánh giá hiện có

```
.\venv\Scripts\python.exe -m unittest discover -s tests -v
python evaluation/run_eval.py `
  --api-url http://localhost:8000 `
  --labels evaluation/labels.real.json `
  --run-name baseline_remeasured
python evaluation/run_benchmark.py `
  --api-url http://localhost:8000 `
  --run-prefix baseline_remeasured --timeout 300
```

Hai lệnh eval chỉ dùng cho schema nhãn hiện tại; sau khi thêm target bbox/split cần adapter/evaluator mới. Không chạy để ghi đè kết quả cũ. Runner histogram/ablation mới do nhóm triển khai, chưa có sẵn chỉ vì được nêu trong tài liệu.

Môi trường venv đang có chạy được 18 test; không đồng nghĩa đã có OpenCV/ultralytics cho chạy detector local. Cài theo service hoặc dùng Docker; không giả định requirements API đã bao gồm toàn bộ dependency detection. Những thay đổi M1/M2 trong thư mục AI cần copy/package rõ vào image detection vì Dockerfile hiện chỉ lấy context object_detection.

Git và dữ liệu

Workspace có thay đổi chưa commit ở search_service và evaluation trước khi lập tài liệu; giữ nguyên, không reset. Chốt snapshot baseline trước khi bắt đầu. Có thể dùng nhánh codex/m1-preprocessing, codex/m2-roi-mask, codex/m3-features, codex/m4-ranking; đây là tên đề xuất, chưa được tạo.

Kiểm tra .gitignore: dữ liệu/results/model có thể bị bỏ qua. Gói nộp vẫn cần nhãn, kết quả và hướng dẫn tải model; không coi có trên máy là người khác clone sẽ có. Review diff trước push; audit dependency trước commit theo hướng dẫn repo, không ghi secrets vào fixture/log.

Sản phẩm cuối và hướng dẫn viết báo cáo

Phân biệt hai tài liệu

Tài liệu này phục vụ giao việc và giữ full context. Báo cáo nộp giảng viên phải viết lại thành bản 4-10 trang; không nộp nguyên tài liệu điều phối thay cho báo cáo học thuật.

Nội dung báo cáo đồ án	Người cung cấp chính	Nội dung cần có
Bài toán và dữ liệu	Cả nhóm	Đầu vào/đầu ra, phạm vi, nguồn, split, nhãn, khó khăn
Tiền xử lý	1	Gamma/EXIF/hình học, điều kiện bật và lý do tham số
Detection và phân vùng	2	YOLO hiện có, mask cổ điển, morphology, giới hạn
Đặc trưng	3	FashionCLIP, HSV, nhánh vô sắc, đồng nhất query/catalog
Retrieval và ranking	4	Đối chứng cổ điển, embedding, hybrid và từ chối
Kết quả và thảo luận	Mỗi người phần mình	Bảng công bằng, ablation, lỗi và chi phí
Phân công và AI	Cả nhóm xác nhận	Công việc thực tế, bằng chứng, công cụ AI và cách kiểm chứng

Mỗi người chuẩn bị khoảng một trang phương pháp/kết quả và hai slide. Nhóm bổ sung slide mở đầu, dữ liệu, metric, kết luận để tổng 10-15 slide. Không ghi đã huấn luyện YOLO/FashionCLIP nếu nhóm chỉ dùng trọng số có sẵn.

Kiểm tra gói nộp

- Có code, requirements theo môi trường, README cài/chạy/đầu ra mong đợi, liên kết tải dữ liệu/trọng số.
- Có 5-10 ảnh mẫu với nguồn/quyền sử dụng; ví dụ thành công và khó; có cấu hình demo ổn định.
- Có nhãn và kết quả cuối, manifest model/catalog/config; người khác chạy lại được hoặc hiểu phần cần quyền truy cập.
- Có PPTX/PDF, báo cáo PDF, khai báo AI và phân công phản ánh việc đã làm.
- ZIP đúng tên lớp/nhóm/đề tài; không chứa .env, credentials, venv/node_modules/cache thừa.
- Video tối đa 3 phút là tùy chọn; ưu tiên hoàn thành thực nghiệm và báo cáo trước.

Checklist khởi động chung

Điền tên bốn thành viên → kiểm tra môi trường → chốt DTO và biên HTTP → kiểm tra baseline/test → phân công ảnh → chốt nhãn/split → mỗi module chạy fixture độc lập → ghép theo từng cặp → validation → khóa cấu hình → test cuối → báo cáo và đóng gói.

Mọi số liệu mới phải có file kết quả. Khi chưa đo, ghi rõ chưa đo. Khi chỉ có phản ví dụ tổng hợp, gọi đúng là phản ví dụ, không suy ra tỷ lệ lỗi ảnh thật.

Nguồn tham chiếu và thuật ngữ

Tài liệu do người dùng cung cấp

Mã	Nguồn và phần dùng
S1	DoAn_XuLyAnh.pdf, 8 trang; yêu cầu đồ án trang PDF 2, 3, 5, 7, 8
S2	2_XLA2_Co ban ve XLA.pdf, 40 trang; lấy mẫu/độ phân giải trang 23-37
S3	3_XLA3_Nang cao chat luong anh (1).pdf, 84 trang; gamma 18-20, histogram 23-38, ngưỡng 39-42
S4	4_XLA4_ Cac he mau va ky thuat in anh.pdf, 66 trang; HSV 37-41
S5	5_XLA5_Phat hien bien.pdf, 100 trang; Canny 36-49, ngưỡng/Otsu 61-70, phát triển vùng 80-85
S6	6_XLA6_Nen anh.pdf, 59 trang; đánh đổi nén 3-4, lượng tử hóa 51-52; chỉ tham khảo phạm vi
S7	7_XLA7_Xu ly hinh thai.pdf, 50 trang; opening/closing 28-38, làm đầy vùng 43-48

Các PDF gốc đang ở C:/Users/Admin/Downloads. Số trang dẫn là thứ tự trang PDF, có thể khác số slide in trên trang. Khi chia sẻ ngoài máy này, gửi tài liệu môn học theo quyền được phép; không phụ thuộc vào đường dẫn cá nhân.

Nguồn dự án được kiểm tra

- **S8:** AI/color_utils.py, embedding_model.py, vectorizer.py; thuật toán màu/encoder/catalog.
- **S9:** object_detection/detector.py, api.py, schemas.py, utils.py; boxes, crop, service/serialization.
- **S10:** api/main.py, search_service.py, schemas.py, detection_client.py; API, ranking và ngưỡng.
- **S11:** db/database.py, docker-compose.yml, Dockerfile và requirements; vector 512, truy xuất/hạ tầng.
- **S12:** evaluation/run_eval.py, run_benchmark.py, build_starter_catalog_eval.py, labels và results/benchmark_current_*.json; phép đo và kết quả cũ.
- **S13:** docs/PHAN_CONG_CAI_TIEN.md; phương án module đã được thống nhất trước báo cáo này.

Các đường dẫn code trên là tương đối với gốc repo Fashion Visual Search. Code và số liệu snapshot được kiểm tra khi lập báo cáo; thuật toán đề xuất chưa được triển khai bởi tài liệu này.

Thuật ngữ cần thống nhất

ROI: vùng ảnh của món đồ. **Mask:** tập pixel được chọn trong ROI. **Embedding:** vector đặc trưng từ encoder. **Descriptor màu:** biểu diễn phân bố màu. **Candidate pool:** tập sản phẩm bước retrieval lấy về. **Rerank:** xếp lại pool. **Ablation:** chỉ thay một thành phần để tách tác dụng. **Validation:** chọn cấu hình. **Test:** đánh giá sau khi khóa. **Fallback:** cách xử lý dự phòng có ghi nhận, không phải bảo đảm dự đoán đúng.